Lecture 9

9.1 Fast Fourier Transform (FFT) 高速フーリエ変換

9.1 Fast Fourier Transform (FFT)

高速フーリエ変換

9.1 Fast Fourier Transform (FFT)

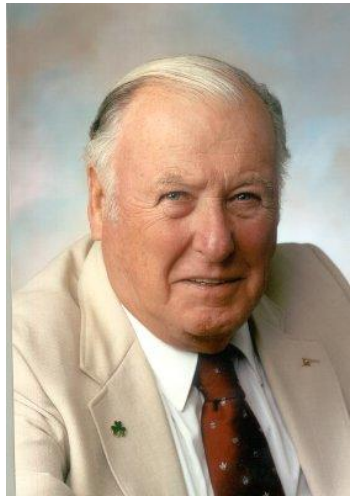
- The Cooley and Tukey Fast Fourier Transform (FFT) algorithm is a turning point to the computation of Discrete Fourier Transform (DFT)
- Before that, DFT was never practical except running by some very expensive computers

Top 10 Algorithms in the 20th Century

1946: The Metropolis Algorithm
1947: Simplex Method
1950: Krylov Subspace Method
1951: The Decompositional Approach to Matrix Computations
1957: The Fortran Optimizing Compiler
1959: QR Algorithm
1962: Quicksort
1965: Fast Fourier Transform
1977: Integer Relation Detection
1987: Fast Multipole Method

9.1 Fast Fourier Transform (FFT)

- The Cooley and Tukey Fast Fourier Transform (FFT) algorithm is a turning point to the computation of Discrete Fourier Transform (DFT)
- Before that, DFT was never practical except running by some very expensive computers



James Cooley

1965: James Cooley of the IBM T.J. Watson Research Center and John Tukey of Princeton University and AT&T Bell Laboratories unveil the **fast Fourier transform**.

Easily the most far-reaching algo-rithm in applied mathematics, the FFT revolutionized signal processing. The underlying idea goes back to Gauss (who needed to calculate orbits of asteroids), but it was the Cooley–Tukey paper that made it clear how easily Fourier transforms can be computed. Like Quicksort, the FFT relies on a divide-and-conquer strategy to reduce an ostensibly $O(N^2)$ chore to an $O(N \log N)$ frolic. But unlike Quick- sort, the implementation is (at first sight) nonintuitive and less than straightforward. This in itself gave computer science an impetus to investigate the inherent complexity of computational problems and algorithms.

John Tukey

[What is computational complexity (計算の複雑さ) and Big-O?]
[1] Big-O notation https://en.wikipedia.org/wiki/Big_O_notation
[2] Big-O notation in 5 minutes -- The basics https://www.youtube.com/watch?v=__vX2sjlpXU

9.1 Fast Fourier Transform (FFT)

Introduction of FFT

Divide and Conquer

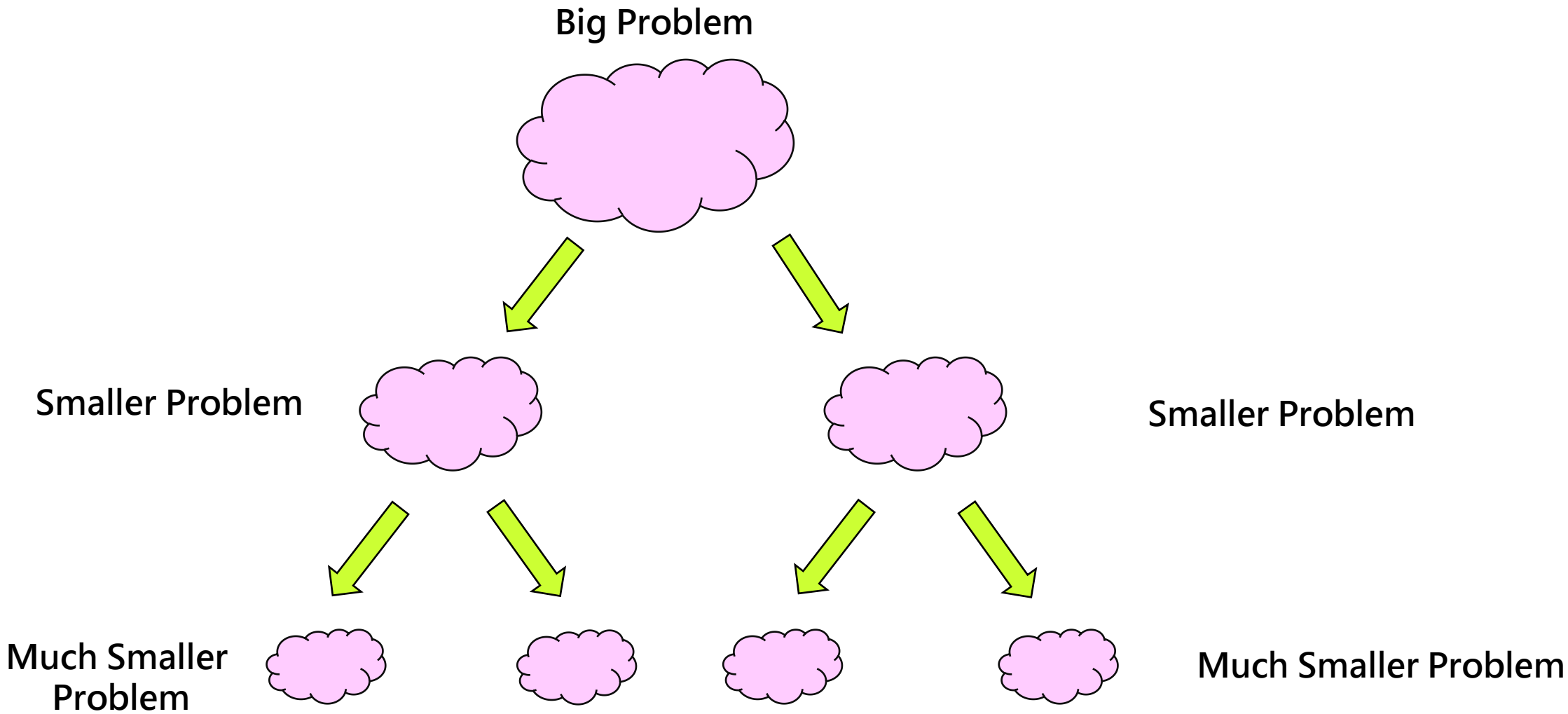
分割統治法

➡ To break down a big problem to a number of smaller problems and tackle them individually

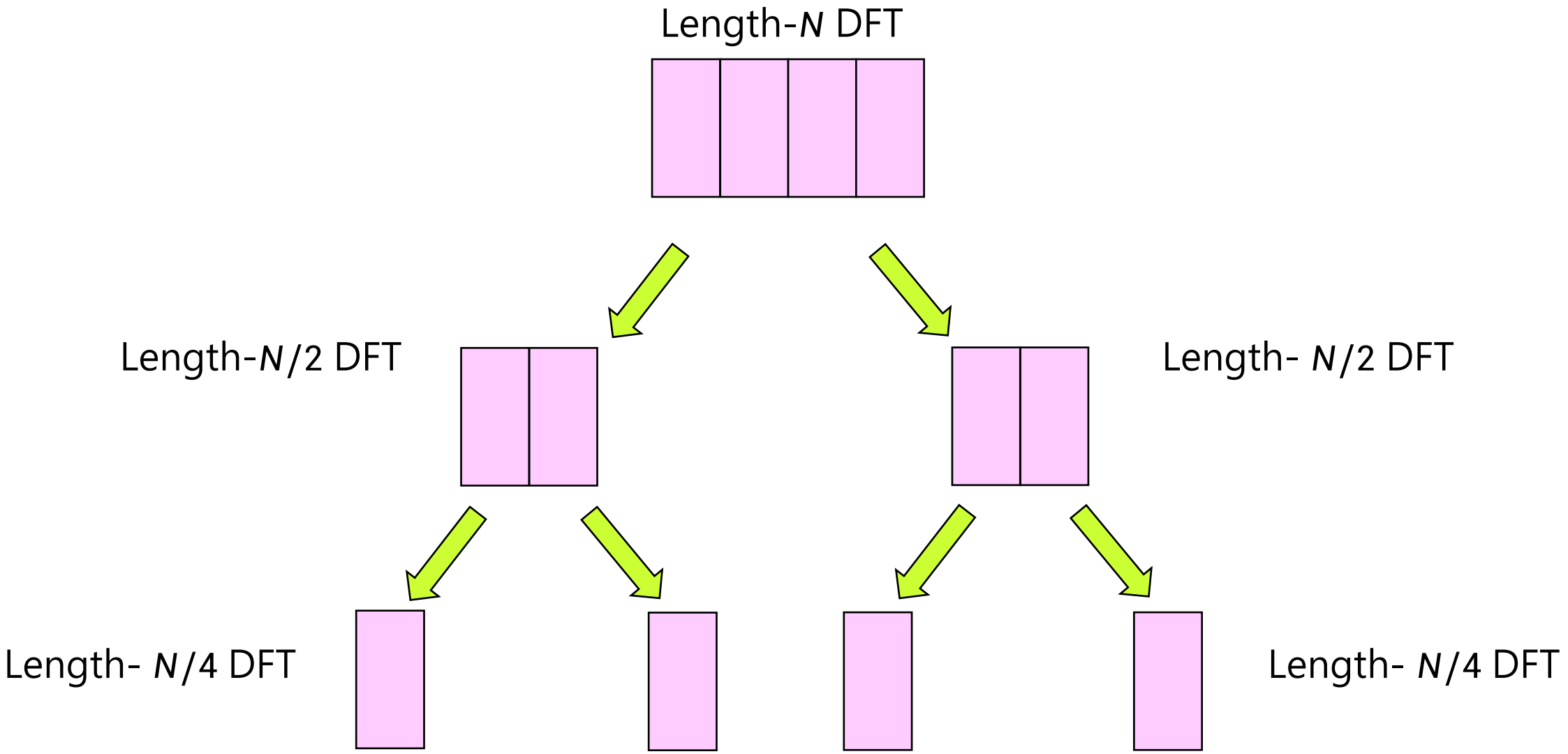
Need to satisfy the following criterion

$$\sum [cost\ (subproblem) + cost\ (overhead)] < cost\ (original\ problem)$$

9.1 Fast Fourier Transform (FFT)



9.1 Fast Fourier Transform (FFT)



9.1 Fast Fourier Transform (FFT)

Let $W_N^{nk} = e^{-i\frac{2\pi}{N}nk}$ (9.1)

Re-writing (9.6) $\hat{f}(k) = \sum_{n=0}^{N-1} f(n)\, e^{-i\frac{2\pi}{N}nk}$ → $\hat{f}(k) = \sum_{n=0}^{N-1} f(n) W_N^{nk}$ (9.2)

It is easy to realize that the same values of W_N^{nk} are calculated many times as the computation proceeds.

Firstly, the integer product nk repeats for different combinations of k and n;

secondly, W_N^{nk} is a periodic function with only N distinct values.

9.1 Fast Fourier Transform (FFT)

Example 9.1 Evaluate W_8^0, W_8^1, W_8^2, W_8^3, W_8^4, W_8^5, W_8^6, W_8^7 for $N = 8$.

(Notice that the FFT is simplest by far if N is an integer power of 2, i.e $N = 4 = 2^2$, $N = 8 = 2^3$).

Solution: By (9.1), we have

$$W_8^0 = e^{-i\frac{2\pi}{8}\cdot 0} = 1$$

$$W_8^1 = e^{-i\frac{2\pi}{8}\cdot 1} = e^{-i\frac{\pi}{4}} = \cos\frac{\pi}{4} - i\sin\frac{\pi}{4} = \frac{\sqrt{2}}{2} - i\frac{\sqrt{2}}{2}$$

$$W_8^2 = e^{-i\frac{2\pi}{8}\cdot 2} = e^{-i\frac{\pi}{2}} = \cos\frac{\pi}{2} - i\sin\frac{\pi}{2} = -i$$

$$W_8^3 = e^{-i\frac{2\pi}{8}\cdot 3} = e^{-i\frac{3\pi}{4}} = \cos\frac{3\pi}{4} - i\sin\frac{3\pi}{4} = -\frac{\sqrt{2}}{2} - i\frac{\sqrt{2}}{2}$$

$$W_8^4 = e^{-i\frac{2\pi}{8}\cdot 4} = -1$$

$$W_8^5 = e^{-i\frac{2\pi}{8}\cdot 5} = -\left(\frac{\sqrt{2}}{2} - i\frac{\sqrt{2}}{2}\right)$$

$$W_8^6 = e^{-i\frac{2\pi}{8}\cdot 6} = i$$

$$W_8^7 = e^{-i\frac{2\pi}{8}\cdot 7} = \frac{\sqrt{2}}{2} + i\frac{\sqrt{2}}{2} = -\left(-\frac{\sqrt{2}}{2} - i\frac{\sqrt{2}}{2}\right)$$

From the above, it can be seen that:

$$W_8^4 = -W_8^0$$

$$W_8^5 = -W_8^1$$

$$W_8^6 = -W_8^2$$

$$W_8^7 = -W_8^3$$

Q: $W_8^8 = ?$

A: $W_8^8 = 1$

9.1 Fast Fourier Transform (FFT)

Also, if nk falls outside the range 0~7, we still get one of the above values:

eg. If $k = 5$ and $n = 7$, $W_8^{5\cdot 7} = W_8^{35} = W_8^{32} \cdot W_8^3 = (W_8^8)^4 \cdot W_8^3 = W_8^3$

9.1 Fast Fourier Transform (FFT)

$$\hat{f}(k) = \sum_{n=0}^{N-1} f(n)\, W_N^{nk} \quad \text{For } k = 0,\ 1,\ \ldots,\ N-1$$

Let us begin by **splitting the single summation over N samples into 2 summations**, each with $\frac{N}{2}$ samples, **one for even** and **the other for odd**.

$$\hat{f}(k) = \sum_{\text{even } n} f(n) W_N^{nk} + \sum_{\text{odd } n} f(n) W_N^{nk}$$

$$\Rightarrow \quad \hat{f}(k) = \sum_{m=0}^{\frac{N}{2}-1} f(2m) W_N^{2mk} + \sum_{m=0}^{\frac{N}{2}-1} f(2m+1) W_N^{(2m+1)k}$$

$$W_N^{(2m+1)k} = W_N^{2mk} W_N^{k}$$

Note that

$$W_N^{2mk} = e^{-i\frac{2\pi}{N}(2mk)} = e^{-i\frac{2\pi}{\frac{N}{2}}mk} = W_{\frac{N}{2}}^{mk}$$

Therefore

$$\Rightarrow \quad \hat{f}(k) = \underbrace{\sum_{m=0}^{\frac{N}{2}-1} f(2m) W_{\frac{N}{2}}^{mk}}_{\frac{N}{2}\text{ - periodic}} + W_N^k \underbrace{\sum_{m=0}^{\frac{N}{2}-1} f(2m+1) W_{\frac{N}{2}}^{mk}}_{\frac{N}{2}\text{ - periodic}} \quad \Rightarrow \quad \text{i.e.} \quad \hat{f}(k) = \hat{f}_{even}(k) + W_N^k \hat{f}_{odd}(k)$$

Overhead

Length-$N/2$ DFT of even data

Length-$N/2$ DFT of odd data

9.1 Fast Fourier Transform (FFT)

Thus the N-point DFT $\hat{f}(k)$ can be obtained from two $\frac{N}{2}$ point transforms, one on even input $\hat{f}_{even}(k)$, and one on odd input $\hat{f}_{odd}(k)$.

Although the frequency index k ranges over N values, only $\frac{N}{2}$ values of $\hat{f}_{even}(k)$ and $\hat{f}_{odd}(k)$ need to be computed since $\hat{f}_{even}(k)$ and $\hat{f}_{odd}(k)$ are periodic in k with period $\frac{N}{2}$.

9.1 Fast Fourier Transform (FFT)

For example, for $N = 8$

- **Even** input data $f(0), f(2), f(4), f(6)$
- **Odd** input data $f(1), f(3), f(5), f(7)$

$$\hat{f}(0) = \hat{f}_{even}(0) + W_8^0 \hat{f}_{odd}(0)$$

$$\hat{f}(1) = \hat{f}_{even}(1) + W_8^1 \hat{f}_{odd}(1)$$

$$\hat{f}(2) = \hat{f}_{even}(2) + W_8^2 \hat{f}_{odd}(2)$$

$$\hat{f}(3) = \hat{f}_{even}(3) + W_8^3 \hat{f}_{odd}(3)$$

$$\hat{f}(4) = \hat{f}_{even}(0) + W_8^4 \hat{f}_{odd}(0) = \hat{f}_{even}(0) - W_8^0 \hat{f}_{odd}(0)$$

$$\hat{f}(5) = \hat{f}_{even}(1) + W_8^5 \hat{f}_{odd}(1) = \hat{f}_{even}(1) - W_8^1 \hat{f}_{odd}(1)$$

$$\hat{f}(6) = \hat{f}_{even}(2) + W_8^6 \hat{f}_{odd}(2) = \hat{f}_{even}(2) - W_8^2 \hat{f}_{odd}(2)$$

$$\hat{f}(7) = \hat{f}_{even}(3) + W_8^7 \hat{f}_{odd}(3) = \hat{f}_{even}(3) - W_8^3 \hat{f}_{odd}(3)$$

$$\hat{f}(k) = \hat{f}_{even}(k) + W_N^k \hat{f}_{odd}(k)$$

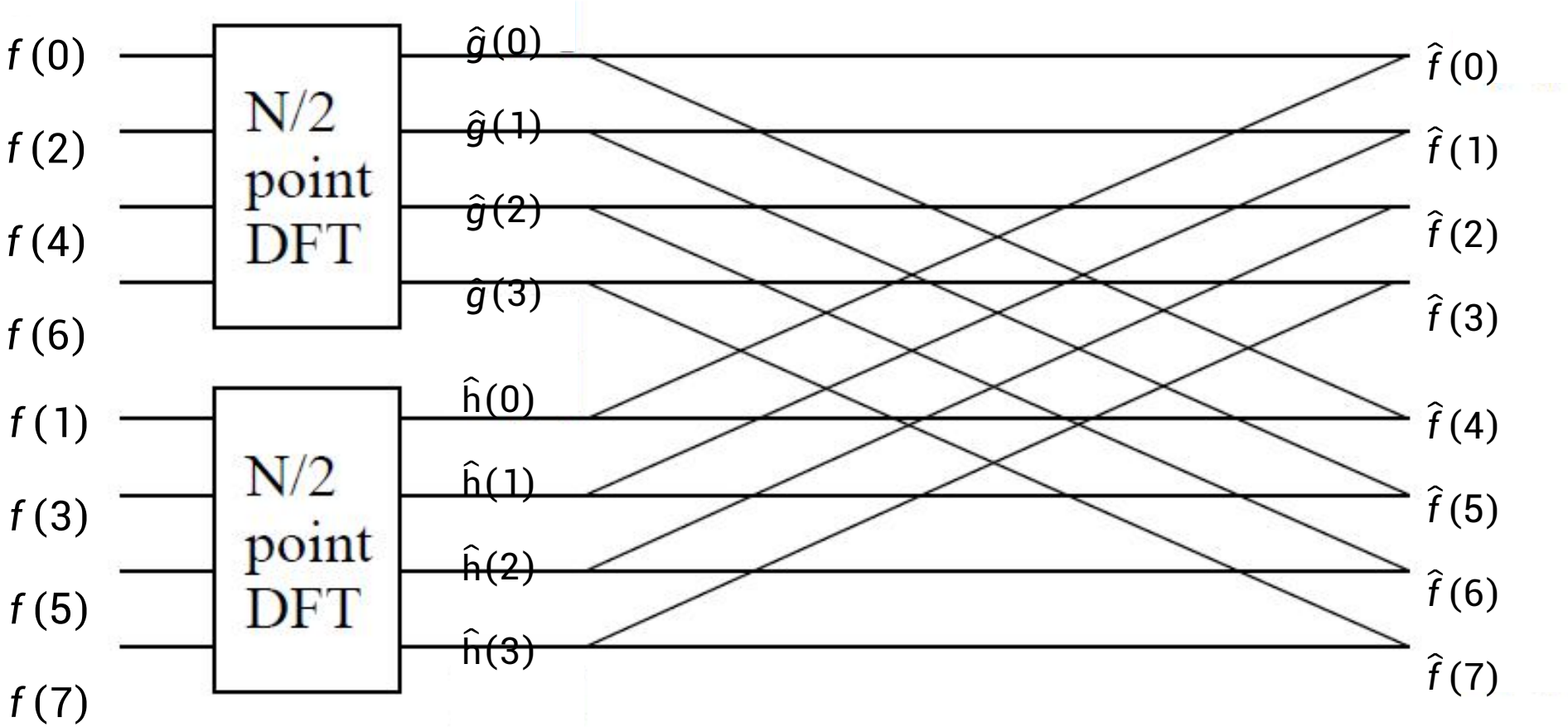
$$\hat{f}\left(k + \frac{N}{2}\right) = \hat{f}_{even}(k) - W_N^k \hat{f}_{odd}(k)$$

Q: How many multiplications in this stage?

A: Only 4 multiplications, i.e. $\frac{N}{2} = \frac{8}{2} = 4$

9.1 Fast Fourier Transform (FFT)

The flow graph of the FFT process for $N = 8$.



Example 9.2 Find the Discrete Fourier Transform by using **Fast Fourier Transform** of the sequence $\{f(0), f(1), f(2), f(3)\} = \{8, 4, 6, 0\}$, $N = 4$.

Solution:

Step 1: Decompose the Sequence

We decompose the sequence $f(x) = \{8,4,6,0\}$ into even and odd indexed subsequences:

Even-indexed subsequence: $f_{even} = \{8,6\}$

Odd-indexed subsequence: $f_{odd} = \{4,0\}$

Step 2: Compute the DFT of Subsequences

Next, we compute the Discrete Fourier Transform (DFT) of these two subsequences. Given their short length, we can compute their DFT directly.

$$\hat{f}(k) = \sum_{n=0}^{N-1} f(n)\ W_N^{nk} \quad \text{For } k = 0,\ 1,\ \ldots,\ N-1 \quad \text{where} \quad W_N^{nk} = e^{-i\frac{2\pi}{N}nk}$$

Solution (Continuous):

Because we handle the half length of the sequence, so the current $N = 2$

$$\hat{f}(k) = \sum_{n=0}^{1} f(n)\ W_2^{nk} \quad \text{For } k = 0,\ 1 \quad \text{where} \quad W_2^{nk} = e^{-i\frac{2\pi}{2}nk}$$

$$W_2^0 = e^{-i\frac{2\pi}{2}0} = 1 \qquad W_2^1 = e^{-i\frac{2\pi}{2}1} = \cos\pi - i\sin\pi = -1$$

Computer DFT for $\hat{f}_{even}(k)$

For $k = 0$

$$\hat{f}_{even}(0) = 8W_2^{0\cdot0} + 6W_2^{1\cdot0} = 14$$

For $k = 1$

$$\hat{f}_{even}(1) = 8W_2^{0\cdot1} + 6W_2^{1\cdot1} = 8 - 6 = 2$$

$$\hat{f}_{even}(k) = \{14,\ 2\}$$

Computer DFT for $\hat{f}_{odd}(k)$

For $k = 0$

$$\hat{f}_{odd}(0) = 4W_2^{0\cdot0} + 0W_2^{1\cdot0} = 4$$

For $k = 1$

$$\hat{f}_{odd}(1) = 4W_2^{0\cdot1} + 0W_2^{1\cdot1} = 4 - 0 = 4$$

$$\hat{f}_{odd}(k) = \{4,\ 4\}$$

Solution (Continuous):

Step 3: Combine the Results

Now we combine the results of the subsequences to form the final FFT result. The combining formula is:

$$\hat{f}(k) = \hat{f}_{even}(k) + W_N^k \hat{f}_{odd}(k) \qquad \hat{f}\left(k + \frac{N}{2}\right) = \hat{f}_{even}(k) - W_N^k \hat{f}_{odd}(k)$$

$$W_4^0 = e^{-i\frac{2\pi}{4}0} = 1 \qquad W_4^1 = e^{-i\frac{2\pi}{4}1} = \cos\frac{\pi}{2} - i\sin\frac{\pi}{2} = -i$$

For $k = 0$ and $k = 2$:

$$\hat{f}(0) = \hat{f}_{even}(0) + W_4^0 \hat{f}_{odd}(0) = 14 + 1 \cdot 4 = 18$$

$$\hat{f}(0+2) = \hat{f}_{even}(0) - W_4^0 \hat{f}_{odd}(0) = 14 - 1 \cdot 4 = 10$$

Solution (Continuous):

For $k = 1$ and $k = 3$:

$$\hat{f}(1) = \hat{f}_{even}(1) + W_4^1 \hat{f}_{odd}(1) = 2 + (-i) \cdot 4 = 2 - 4i$$

$$\hat{f}(1+2) = \hat{f}_{even}(1) - W_4^1 \hat{f}_{odd}(1) = 2 - (-i) \cdot 4 = 2 + 4i$$

Therefore, the final FFT result is

$$\hat{f}(k) = \{18,\ 2 - 4i,\ 10,\ 2 + 4i\}$$

9.1 Fast Fourier Transform (FFT)

Moreover,

Assuming that N is a power of 2 (i.e. there are p stages, where $N = 2^p$), we can repeat the above process on the two $\frac{N}{2}$ point transforms, breaking them down to $\frac{N}{4}$ point transforms, etc ..., until we come down to 2-point transforms.

Thus the FFT is computed by dividing up, or decimating, the sample sequence $\hat{f}(k)$ into sub-sequences until only 2-point DFT's remain. Since it is the input, or time, samples which are divided up, this algorithm is known as the *decimation-in-time* (DIT) algorithm.

9.1 Fast Fourier Transform (FFT)

Computational Speed of FFT

- The DFT requires N^2 complex multiplications.
- At each stage of the FFT (i.e. each halving) $\frac{N}{2}$ complex multiplications are required to combine the results of the previous stage.
- Since there are $\log_2 N$ stages, the number of complex multiplications required to evaluate an N-point DFT with the FFT is approximately $\frac{N}{2}\log_2 N$ (approximately because multiplications by factors such as W_N^0, $W_N^{\frac{N}{2}}$, $W_N^{\frac{N}{4}}$, $W_N^{\frac{3N}{4}}$ and are really just complex additions and subtractions).

N	N^2 (DFT)	$\frac{N}{2}\log_2 N$ (FFT)	Computation Cost Saving
32	1,024	80	92%
256	65,536	1024	98%
1,024	1,048,576	5120	99.5%

9.1 Fast Fourier Transform (FFT)

Q: What if N is not a power of 2?

Usually, implement padding for the data with zeroes.

e.g. Assume we have data with 5 (i.e. $5 = 2^2 + 1$) points, then we perform the **padding** that include 3 zeroes with the 8 points (i.e. $N = 5 + 3 = 8 = 2^3$) and compute a 8-point FFT.

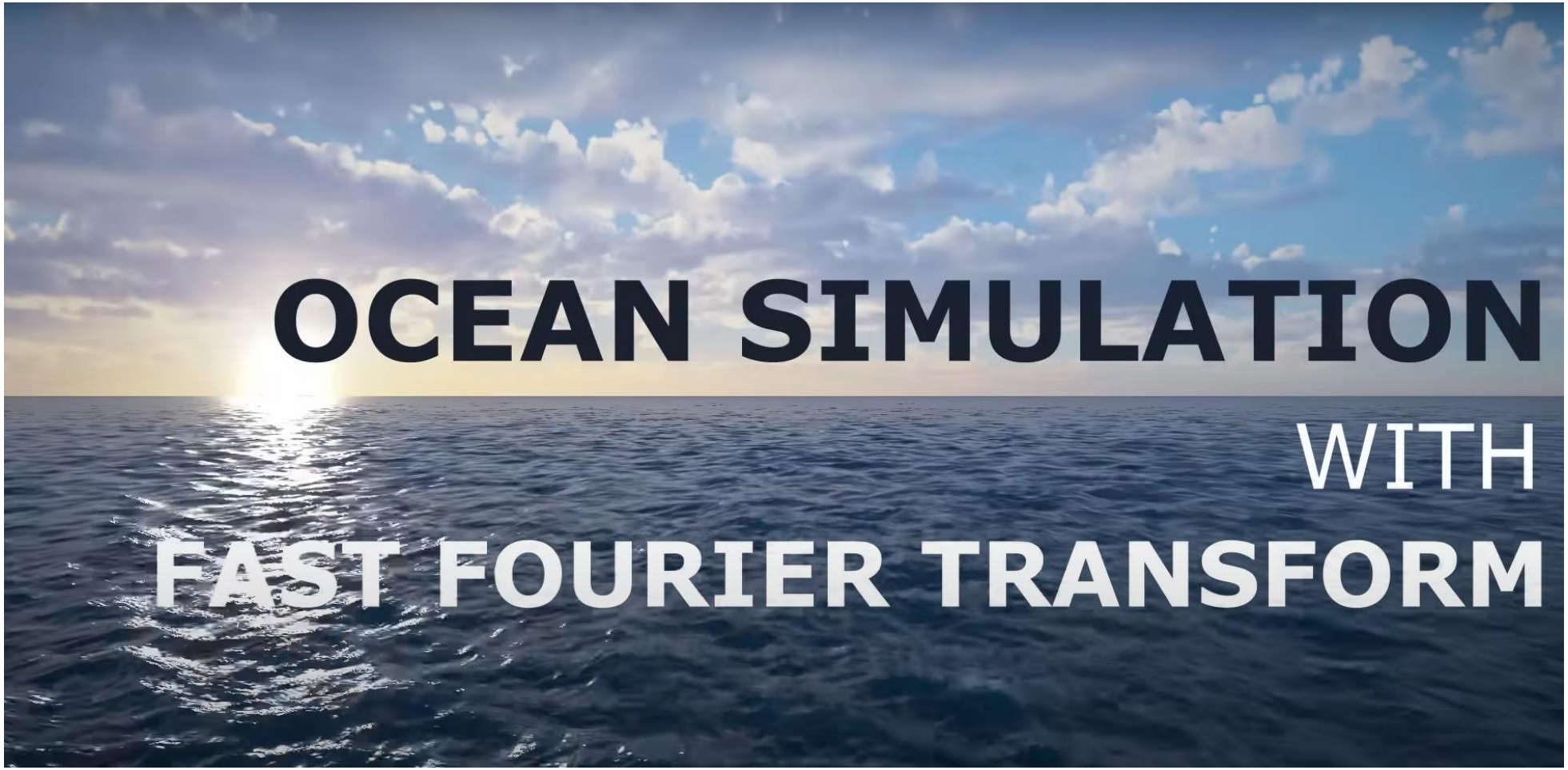
$f(n) = \{0,1,0,1,1\}$ $\quad N = 5 = 2^2 + 1$

After **zero padding** $\quad f(n) = \{0,1,0,1,1,0,0,0\}$ $\quad N = 8 = 2^3$

Remarks

- FFT requires N to be a power of 2.
- If N is not a power of 2, need zero padding to let N be a power of 2 before FFT.

9.1 Fast Fourier Transform (FFT)



https://www.youtube.com/watch?v=kGEqaX4Y4bQ

Review for Lecture 9

- Computational Complexity (Big-O)
- Fast Fourier Transform (FFT)

Assignment

Please Check https://web-ext.u-aizu.ac.jp/~xiangli/teaching/MA05/index.html

Reading materials: Lecture 9 Slides

References

[1] Big-O notation https://en.wikipedia.org/wiki/Big_O_notation

[2] Big-O notation in 5 minutes -- The basics https://www.youtube.com/watch?v=__vX2sjlpXU&t=238s

[3] Fast Fourier Transform How to implement the Fast Fourier Transform algorithm in Python from scratch. https://towardsdatascience.com/fast-fourier-transform-937926e591cb

[4] Daniel P. K. Lun, http://www.eie.polyu.edu.hk/~enpklun/EIE327/FFT.pdf

[5] Alex Townsend, http://pi.math.cornell.edu/~ajt/presentations/TopTenAlgorithms.pdf

[6] Barry A. Cipra, http://www.uta.edu/faculty/rcli/TopTen/topten.pdf

[7] Nakhlé H. Asmar, *Partial Differential Equations with Fourier Series and Boundary Value Problems 2nd Edition*, 2004